Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SangramBhosale>redis
'redis' is not recognized as an internal or external command, operable program or batch file.

C:\Users\SangramBhosale>redis-server
[17044] 25 Mar 12:24:09.840 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
[17044] 25 Mar 12:24:09.840 # Redis version=5.0.14.1, bits=64, commit=ec77f72d, modified=0, pid=17044, just started
[17044] 25 Mar 12:24:09.840 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
[17044] 25 Mar 12:24:09.851 # Could not create server TCP listening socket *:6379: bind: An operation was attempted on something that is not a socket.

C:\Users\SangramBhosale>
how to run it

You're **very close** — the important part is:

✅ `redis-server` command **is installed and recognized**

❌ But Redis **cannot bind to port 6379** (or Windows networking issue with this Redis build)

This line is the key:

```
TCP listening socket *:6379: bind: An operation was attempted on something that is not a socket
```

bat

Why this happens

Usually on Windows, this happens because **one of these is true**:

1. **Port 6379 is already being used** by another Redis/service
2. **This Windows Redis build is old/quirky** (common with Redis 5 Windows ports)
3. **IPv6 / socket binding issue** on Windows
4. You started Redis from a bad/unsupported build

Best fix (quickest): Run Redis on another port

Try this first:

bat

If it starts successfully, you'll see something like:

```bat
```

Then update your Django `.env` or settings:

```env
```

If your Celery config uses broker/result backend:

```python
CELERY_BROKER_URL = os.getenv("REDIS_URL", "redis://127.0.0.1:6380/0")
```

Run

👉 This is the easiest solution.

Step-by-step proper check (recommended)

Step 1) Check if port 6379 is already in use

Run:

```bat
```

If you see output like:

```bat
```

That means **something is already using 6379**.

Then find what process it is:

```bat
```

If it's already Redis, then Redis is already running 😄
In that case, **you don't need to start another one**.

Step 2) If Redis is already running, test it

Try:

```bat
```

If it says:

```bat
```

✅ Redis is already working.

Then keep your `.env` as:

```env
```

Step 3) If 6379 is blocked, use 6380

Start Redis:

```bat
```

Then in a **new terminal**, test it:

```bat
```

Expected:

```bat
```

If `redis-cli` is not recognized

Sometimes only `redis-server` works from PATH.

Try this in the Redis install folder, or use full path, for example:

```bat
```

(Adjust path if different.)

If `redis-server --port 6380` still fails

Then use this safer command:

```bat
```

This forces Redis to bind only to localhost.

If still failing, try:

```bat
```

Best Windows recommendation (important)

Since you're on **Windows**, old native Redis builds can be unstable.

Better options:

1. **Memurai** (best native Windows Redis-compatible)
2. **Docker Redis**
3. **WSL + Redis**

Since you already need this for your Django + Celery project, **Memurai or Docker** is usually much smoother.

For your current project (what you should do now)

Use this exact flow:

Terminal 1 – Start Redis

Try:

```bat
```

Leave this terminal open.

Terminal 2 – Start Celery worker

Inside your project virtualenv:

```bat
```

Terminal 3 – Start Celery beat

```bat
```

Terminal 4 – Start Django

```bat
```

Update `.env`

Use:

```env
```

If you want, I can now give you the exact final working `settings.py` + `celery.py` + `.env` configuration for your Wellness project (copy-paste ready for Windows + port 6380), so you can run it without guessing.

[2026-03-25 12:42:14,159: INFO/MainProcess] Task core.tasks.run_presence_check_task[f21e234a-c439-4b0a-9fb3-7e627f1b4ad6] succeeded in 14.156999999999243s: None
[2026-03-25 12:44:00,009: INFO/MainProcess] Task core.tasks.reset_daily_trackers_task[0418b255-0325-4bfc-b88d-ca05218b9a5b] received
[2026-03-25 12:44:00,214: WARNING/MainProcess] Deleted old PresenceLog records: 0
[2026-03-25 12:44:00,214: WARNING/MainProcess] Deleted old GoodMorningLog records: 0
[2026-03-25 12:44:00,215: WARNING/MainProcess] Reset 15 activity trackers
response in worker

[2026-03-25 12:44:00,001: INFO/MainProcess] Scheduler: Sending due task reset-daily-trackers (core.tasks.reset_daily_trackers_task)
in celery beat why it did not delete logs

This is actually **working correctly** ✅

Your worker log says:

```
Deleted old PresenceLog records: 0
Deleted old GoodMorningLog records: 0
Reset 15 activity trackers
```

That means:

- Celery Beat **did trigger the task**
- Worker **did receive and execute it**
- The delete query **ran successfully**
- But there were **no old records to delete**

Why it deleted `0` logs

Your function logic is likely this (from your earlier code):

```
today = timezone.localdate()

PresenceLog.objects.exclude(
    checked_at__date=today
).delete()

GoodMorningLog.objects.exclude(
    sent_at__date=today
).delete()
```

This means:

> **Delete only records NOT from today**

So if all your logs are from **today (March 25)**, then:

- nothing is “old”
- therefore deleted count = **0**

Important: Your task ran at 12:44 PM, not 7:55 AM

From your log:

```
[2026-03-25 12:44:00] Scheduler: Sending due task reset-daily-trackers
```

So today the task ran at **12:44 PM**.

If all PresenceLog / GoodMorningLog entries were created **today before 12:44 PM**, they are still considered **today’s logs**, so they are **NOT deleted**.

That is expected.

What your current logic does

Your current logic is:

- Keep **today's logs**
- Delete only **yesterday and older**
- Reset activity trackers for all employees

So at any time during today:

- today's logs stay
- older logs are removed

This is exactly why count is `0` if there were no logs from previous days.

How to verify it properly

Check database records in Django shell

Run:

```bash
```

Then:

```
from core.models import PresenceLog, GoodMorningLog
from django.utils import timezone

today = timezone.localdate()

print("Today:", today)
print("Presence total:", PresenceLog.objects.count())
print("Presence old:", PresenceLog.objects.exclude(checked_at__date=today).count())

print("GoodMorning total:", GoodMorningLog.objects.count())
print("GoodMorning old:", GoodMorningLog.objects.exclude(sent_at__date=today).count())
```

If output shows:

```python
Presence old: 0
```

Python Run

👉 then your task is behaving perfectly.

Most likely reason in your case

Because you recently started testing Celery today, all logs are probably:

- created **today**
- therefore not “old”
- therefore not deleted

So the result:

```
Deleted old PresenceLog records: 0
Deleted old GoodMorningLog records: 0
```

is **correct**.

If you want to test deletion right now

Option 1 (best): Create a fake old record manually

In Django shell:

```python
from core.models import PresenceLog
from core.models import Employee
from django.utils import timezone
from datetime import timedelta

emp = Employee.objects.first()

log = PresenceLog.objects.create(
    employee=emp,
    status="Available",   # use your actual required fields
)
```

Then make it old:

```python
log.checked_at = timezone.now() - timedelta(days=1)
```

Python Run

Now run your task manually:

```python
from core.tasks import reset_daily_trackers_task
```

Python Run

Then worker should show something like:

```
Deleted old PresenceLog records: 1
```

One more subtle thing: `.delete()` return value

If your code is:

```python
deleted_presence_count, _ = PresenceLog.objects.exclude(
    checked_at__date=today
```

Python Run

That `deleted_presence_count` is:

- total rows deleted including cascades (if any)
- so if model relationships delete extra child rows, count can be bigger

But if it shows `0`, that means **literally nothing matched**.

Very important check: field names

Make sure you are using the correct datetime fields:

- `PresenceLog` → `checked_at`
- `GoodMorningLog` → maybe `sent_at` or `created_at`

If you accidentally used wrong field names in query, Django would usually crash with a field error — and your task would fail.
Since task is succeeding, field names are probably correct.

Why Beat log + Worker log together prove it's fine

You showed:

Beat:

```
Scheduler: Sending due task reset-daily-trackers
```

Worker:

```
Task core.tasks.reset_daily_trackers_task received
Deleted old PresenceLog records: 0
Deleted old GoodMorningLog records: 0
Reset 15 activity trackers
```

This confirms:

- ✅ Beat schedule is working
- ✅ Worker is connected
- ✅ Task registration is correct
- ✅ Redis broker is working
- ✅ Database write is working
- ✅ Reset logic is running

So your **Celery setup is now functioning** 🎉

My conclusion for your Wellness project

At this point, your Celery migration looks **successfully working**.

The only reason logs weren't deleted is:

> **There were no previous-day logs to delete.**

Best production-safe version of your reset function

Use this (recommended):

```
import logging
from django.utils import timezone
from core.models import PresenceLog, GoodMorningLog, ActivityTracker

logger = logging.getLogger(__name__)


def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs and GoodMorningLogs
    so only today's logs remain.
    """
    today = timezone.localdate()

    # Delete old presence logs
    deleted_presence_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Delete old good morning logs
    deleted_good_morning_count, _ = GoodMorningLog.objects.exclude(
        sent_at__date=today
    ).delete()
```

```
    # Reset trackers
    updated_count = ActivityTracker.objects.update(
        inactive_count=0,
        good_morning_sent=False,
        reminder_count=0,
    )

    logger.warning(f"Deleted old PresenceLog records: {deleted_presence_count}")
    logger.warning(f"Deleted old GoodMorningLog records: {deleted_good_morning_count}")
    logger.warning(f"Reset {updated_count} activity trackers")
```

Next thing you should do now (important)

Since your Celery infra is working, now test **manual task execution**:

Bash

```
from core.tasks import run_presence_check_task
```

Python Run

If worker receives it and processes reminders properly, then your whole migration is good.

If you want, send me your **exact current** `reset_daily_trackers()` **function +** `tasks.py` **+ celery beat schedule**, and I'll verify **100% whether your daily reset and 30-min presence checks are correctly**

configured (line by line, no guessing).

oh thats the case there are no old logs thats why it didnt delete it but when i deleted theose files and restarted all servers again it worked right

Yes — **exactly right** ✅

What happened is:

- Earlier, there were **no old logs** in DB
  → so task ran, but **deleted 0**
- Then you **manually deleted those records / restarted services**
- After restart, Celery + Beat + Worker still ran correctly
- So your setup is **working properly**

In short:

Yes, it worked correctly. 🎉

What this proves in your project

Your current setup is now doing all of this correctly:

- ✅ **Redis is working**

- ✅ **Celery worker is running**
- ✅ **Celery beat is running**
- ✅ **Beat is sending scheduled tasks**
- ✅ **Worker is receiving tasks**
- ✅ **Django DB operations are executing**
- ✅ `reset_daily_trackers_task` **is working**
- ✅ `run_presence_check_task` **is also running** (you already showed success logs)

So for your **Wellness Reminder project**, the **APScheduler → Celery migration is basically successful now**.

Very important note for your testing

When you restart all servers, always start in this order on Windows:

1) Redis

```bat
```

2) Celery worker

```bat
```

3) Celery beat

```bat
```

4) Django server

```bat
```

👉 This order avoids many random issues during local testing on Windows.

My honest conclusion

Yes Sangram — your Celery setup is now working correctly.
At this point, the remaining work is mostly:

- verifying **schedule times**
- verifying **message sending logic**
- verifying **duplicate prevention**
- preparing for **deployment**

Next best step (recommended)

Now test these 3 things one by one:

1) Manual presence check task

In Django shell:

```python
from core.tasks import run_presence_check_task
```

Python Run

Confirm worker receives it.

---

2) Beat schedule timing

Make sure:

- presence check runs every **30 min**
- daily reset runs **once at 7:55 AM Mon–Fri**

---

3) Teams proactive message test

Confirm reminders are sent only when:

- user is inactive / away / offline (your exact logic)
- not spammed repeatedly
- daily counters reset properly

Short answer

Yes, 100% — it worked.

The `0 deleted` was only because **there were no old logs**.

Your Celery + Beat + Worker pipeline is functioning now. ✅

If you want, I can now do the **final production check** for your project and give you:

“Final clean Celery setup for Wellness Reminder”

- `celery.py`
- `__init__.py`
- `settings.py`
- `tasks.py`
- `CELERY_BEAT_SCHEDULE`
- exact Windows run commands
- common mistakes to avoid

👉 Send `settings.py` + `celery.py` + `core/tasks.py`, and I’ll review the whole setup line-by-line.